

Problem Set 5

A. Hierarchy

TIANZHEYU SETTINGS LISTS BLOG TEAMS SUBMISSIONS CONTESTS

☒ show unofficial

tianzheyu submissions

tianzheyu submissions							
#	When	Who	Problem	Lang	Verdict	Time	Memory
381573331	Jul/07/2026 18:01 UTC+10	tianzheyu	B - Hierarchy	C++23 (GCC 14-64, msys2)	Accepted	124 ms	0 KB

<https://codeforces.com/submissions/tianzheyu#>

Process

The question is saying that there are n employees each except one with exactly one supervisor. Each employee has a specific qualification. And there are m applications (a, b, c), such that employee a is the supervisor of employee with c cost. For each application, there is always supervisor's qualification is larger than the qualification of employee under management. The question aimed to find the minimum cost of such a hierarchy, or find out that it is impossible to build it.

Challenges and Overcoming

The question essentially asks us to find the directed minimum spanning tree in that DAG. After getting some hint, generally, finding this in a directed graph requires the Chu-Liu/Edmonds algorithm. However, the hierarchy is a DAG since the supervisor's qualification is always larger. Because of this DAG property, we can bypass the complex algorithm and use a greedy approach. We can find the minimum spanning tree just by selecting the in edge with smallest cost for each node. And after that, we will check if in the graph there is only 1 node having no in edge (by define there is one node without supervisor).

However, when coding this greedy approach, my program initially failed on test cases where the hierarchy was impossible to build. I found this bug when my program outputted a massive garbage sum instead of -1.

Tracing back the logic, the issue was how I handled unreachable nodes. I had initialized the `min_in_cost` array with a large infinity value ($1e9$) to find the minimum edges. But in my final summation loop, I forgot to check if a node actually received a valid supervisor.

Next time to prevent this bug, whenever I use an INF initialization for a greedy choice array, I will explicitly write a validation pass before summing the costs.